Dinamičko testiranje



Autor: Goran Savić

* slajdovi su kreirani na osnovu knjige *Spillner, A., Linz, T., Schaefer, H.,* ***Software Testing Foundations****, 4th Edition, Rocky Nook, 2014*

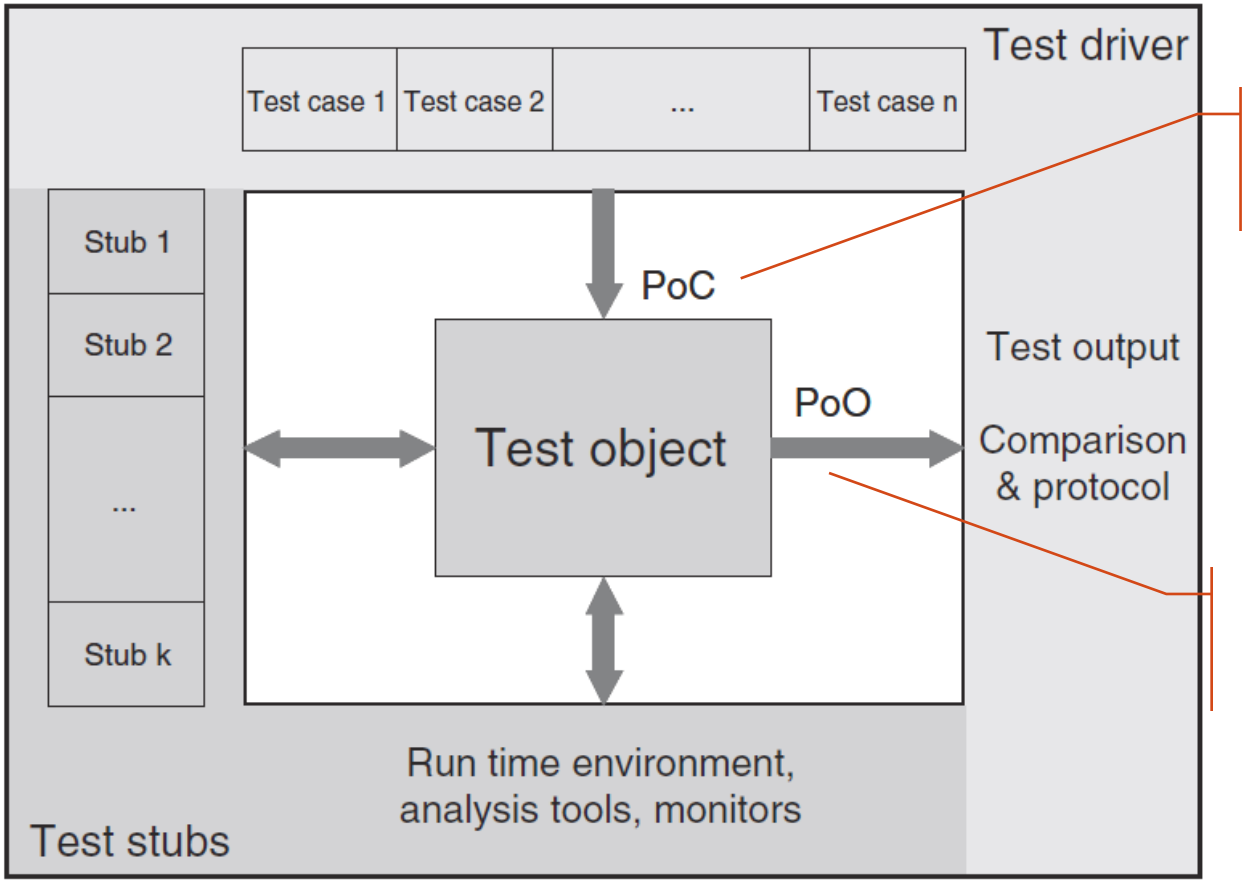
Dinamičko testiranje

- Dinamičko testiranje predstavlja verifikaciju ponašanja test objekta putem izvršavanja njegovih funkcionalnosti
  - test objekat izvršava programiranu funkcionalnost za ulazne podatke koje mu šaljemo
  - test objekat mora da bude izvršiv

Izvršavanje test objekta

- U nižim nivoima testiranja (jedinično i integraciono) potrebno je obezbediti izvršivost test objekta
  - test objekat se izvršava u test okruženju
    - naziva se *test bed, test harness, test environment*
    - obezbeđuje softversko i hardversko okruženje za izvršavanje
  - treba simulirati ponašanje objekata koje test objekat koristi
    - zato što ti objekti još nisu implementirani ili
    - zato što želimo izolovano testiranje test objekta
  - treba omogućiti izvršavanje test objekta za potrebe testiranja (nezavisno od pokretanja aplikacije)
    - za ovo je zadužen *test driver*, koji pokreće kod koji se izvršava nad test objektom (npr. JUnit)

Komponente dinamičkog testiranja



Tačka upravljanja
Point of Control (PoC)

Tačka posmatranja
Point of Observation (PoO)

Cilj dinamičkog testiranja

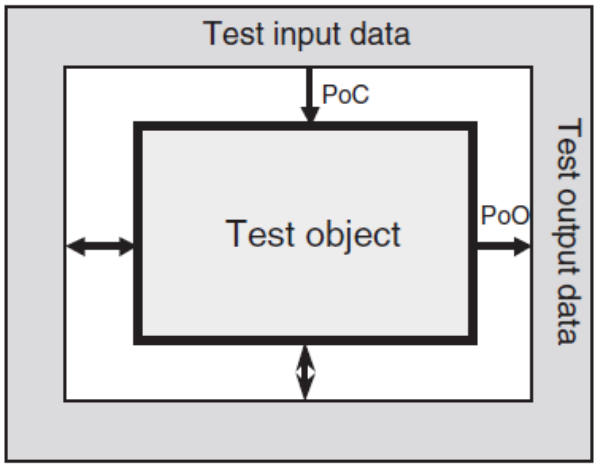
- Cilj je
  - da se pokaže da testirani objekat zadovoljava funkcionalnosti
  - da se inicira pojava otkaza testiranog objekta za funkcionalnosti čija implementacija sadrži nedostatke
- Dizajn test slučaja
  - utvrditi preduslove i ciljeve testa
  - definisati test slučaj
  - implementirati test slučaj u programskom jeziku
  - odrediti način izvršavanja slučaja (redosled, grupisanje, ...)

Tipovi dinamičkog testiranja

- *Black-box testing*
- *White-box testing*
- Testiranje zasnovano na iskustvu
  - *experience-based testing*

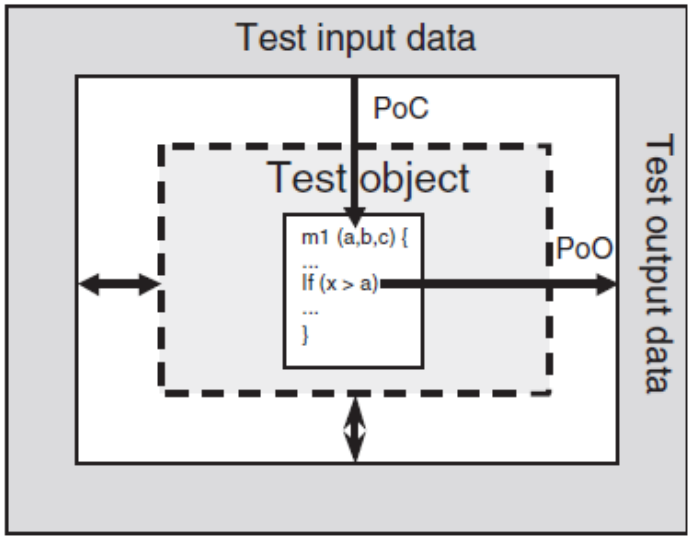
Black box testing

- Test objekat se posmatra kao crna kutija
- Test slučajevi se kreiraju na osnovu specifikacije objekta
  - nije neophodno analizirati i strukturu objekta
- Ponašanje objekta se posmatra „spolja“
  - kakve izlaze daje za definisane ulaze
  - tačka upravljanja kao i tačka posmatranja rezultata se nalaze izvan objekta



White box testing

- testovi se dizajniraju na osnovu uvida u strukturu test objekta
  - analizira se programski kod objekta
- Testiranje analizira i tok programa unutar test objekta
  - Tačka posmatranja je unutar objekta
  - Tačka upravljanja može biti unutar objekta kada se objekat modifikuje za potrebe testiranja da bi se izvršio željeni tok programa



Black box vs white box testing

- *white box* se koristi u testovima nižeg nivoa
  - jedinični i integracioni testovi
- *black box* se koristi za e2e testove
  - može i za integracione
- *black box* je isključivi način testiranja kod tehnika koje kreću razvoj programa od testova
  - *test driven development*
- *gray box* tehnike
  - Tehnike testiranja koje uključuju principe i *white box* i *black box* testiranja

Testiranje zasnovano na iskustvu i intuiciji

- Koristi znanje i veštine učesnika u projektu da bi se kreirali test slučajevi za koje se smatra da će najefikasnije verifikovati program i ukazati na nedostatke
  - nije sistematična tehnika
  - oslanja se na znanje o tehnologiji, kao i iskustvo iz ranijih projekata da bi identifikovalo otkaze

Black box testiranje



Black box testing

- Ne bavi se unutrašnjom strukturom i dizajnom test objekta
- Test se kreira na osnovu specifikacije test objekta
  - ova tehnika se naziva i testiranje zasnovano na specifikaciji ili na zahtevima
- Proverava ponašanje test objekta za zadate ulaze
- Kompletan test bi trebao da testira ponašanje objekta za sve moguće ulaze
  - ovo nije moguće realizovati
  - zato se koriste metode za dizajn test slučajeva izborom podskupa svih mogućih kombinacija ulaznih podataka

Metode *black-box* testiranja

- Podela na klase ekvivalencije
- Analiza graničnih vrednosti
- Testiranje prelaza stanja
- Logički zasnovane tehnike
- Testiranje zasnovano na slučajevima korišćenja

Podela na klase ekvivalencije



Podela na klase ekvivalencije

- Domen mogućih vrednosti za svaki ulazni parametar se deli na klase ekvivalencije
- Klasa ekvivalencije
  - skup vrednosti za koje se pretpostavlja da ih test objekat obrađuje na isti način
- Dovoljno je testirati jednu od vrednosti iz klase ekvivalencije
  - smatra se da se test objekat isto ponaša za ostale vrednosti iz iste klase ekvivalencije
- Potrebno je utvrditi klase ekvivalencije kako za ispravne, tako i za neispravne ulaze

Primer klasa ekvivalencije

- Metoda koja izračunava ocenu studenta na osnovu broja bodova
- Validne klase ekvivalencije (vEC)

Parametar	Klasa ekvivalencije	Reprezentativna vrednost
Broj bodova	vEC1: $95 \leq x \leq 100$	97
	vEC2: $85 \leq x \leq 94$	92
	vEC3: $75 \leq x \leq 84$	78
	vEC4: $65 \leq x \leq 74$	71
	vEC5: $55 \leq x \leq 64$	63
	vEC6: $0 \leq x \leq 54$	43



- Primer
  - `students-testing/StudentServiceTest::testCalculateGrade()`

Primer klasa ekvivalencije

- Nevalidne klase ekvivalencije (iEC)

Parametar	Klasa ekvivalencije	Reprezentativna vrednost
Broj bodova	iEC1: x < 0	-17
	iEC2: x > 100	192



- Primer
  - `students-testing/StudentServiceTest::testCalculateGradeNegative1()`
  - `students-testing/StudentServiceTest::testCalculateGradeNegative2()`

Kreiranja klasa ekvivalencije

- Treba biti veoma oprezan pri kreiranju klasa ekvivalencije
  - pretpostavlja se da se za sve vrednosti u jednoj klasi sistem isto ponaša
  - česta greška je da se za određenu vrednost ne identifikuje da pripada posebnoj klasi ekvivalencije
    - pogrešno se pretpostavi da se sistem za tu vrednost ponaša na isti način kao za druge vrednosti
- Posebno voditi računa o vrednostima na granici klase ekvivalencije
  - time se detaljno bavi *black box testing* metoda „analiza graničnih slučajeva“

Klase ekvivalencije izlaznih parametara

- Umesto ulaznih, možemo klasifikovati izlazne parametre
  - identifikuju se moguće vrednosti izlaza
  - klasifikuju se u klase ekvivalencije
  - za svaku klasu ekvivalencije utvrđuje se koja kombinacija ulaznih parametara rezultuje izlaznom vrednošću iz te klase

Dizajniranje test slučajeva

- Za svaki ulazni podatak utvrđuju se domen ispravnih vrednosti i domen neispravnih vrednosti
- Time dobijamo dve klase ekvivalencije
- Svaka od ove dve klase ekvivalencije se dalje deli na klase ekvivalencije
  - na osnovu toga kako program procesira različite vrednosti ulaznih podataka
- Deljenje na klase ekvivalencije se završava kada svakom zahtevu odgovara jedna klasa ekvivalencije
- Zatim se za svaku klasu ekvivalencije bira reprezentativna vrednost
- Konačno, potrebno je utvrditi preduslove i očekivani rezultat za svaki test slučaj

Dizajniranje test slučajeva

- Test objekat obično očekuje više ulaznih parametara
- Za svaki parametar postoji više klasa ekvivalencije
- Da bi test bio što pouzdaniji
  - reprezentativne vrednosti iz svih validnih klasa ekvivalencije bi trebale biti kombinovane za sve ulazne parametre
    - da budu pokrivene sve moguće kombinacije validnih klasa ekvivalencije
    - time se dobijaju pozitivni test slučajevi
  - reprezentativna vrednost iz nevalidne klase ekvivalencije bi trebala biti kombinovana samo sa reprezentativnim vrednostima iz validnih klasa ekvivalencije

Dizajniranje test slučajeva

- Pomenuta pravila bi zahtevala kreiranje prevelikog broja pozitivnih test slučajeva
  - jer se formira dekartov proizvod između svih klasa ekvivalencija svakog od parametara
- Zato se vrši redukcija broja test slučajeva
  - vrši se analiza prioriteta, pa se kreiraju samo relevantni test slučajevi
    - sadrže kombinacije ulaznih parametara koje se često pojavljuju
  - poželjni su test slučajevi koje uključuju granične slučajeve
  - napraviti sve moguće kombinacije na nivou parova parametara, ali ne i svih parametara
  - minimalno podržati da se reprezentativna vrednost svake klase ekvivalencije nalazi u bar jednom test slučaju

Dizajniranje test slučajeva

- Ne treba kombinovati reprezentativne vrednosti iz nevalidne klase ekvivalencije sa vrednostima iz drugih nevalidnih klasa ekvivalencije
  - već jedna nevalidna vrednost bi trebala da izazove izuzetak
  - ako u test slučaju imamo više nevalidnih vrednosti, time se sakriva kako sistem reaguje na svaku od njih

Kriterijum završetka

- Može biti definisan kao procenat izvršenih klasa ekvivalencije u odnosu na ukupan broj identifikovanih klasa ekvivalencije
- $coverage = \frac{testedEC}{totalEC} * 100\%$

- Neispravno utvrđen premali broj klasa ekvivalencije može da rezultuje visokom *coverage* vrednošću
  - visoka vrednost tada lažno ukazuje da je softver dobro testiran

Pregled korišćenja klasa ekvivalencije

- Sistematična tehnika
- Smanjuje broj potrebnih test slučajeva
  - kroz klase ekvivalencije se identifikuju ulazne vrednosti koje rezultuju istim ponašanjem
  - za sve vrednosti iz iste klase ekvivalencije kreira se samo jedan test slučaj
- Metodu je moguće koristiti i u kombinaciji sa drugim tehnikama *black box* testiranja
  - u nastavku je korišćenje sa „analizom graničnih slučajeva“
- Nedostatak je što metoda klasifikuje samo vrednosti pojedinačnih parametara
  - međusobna zavisnost između parametara se ignoriše
  - nekada sistem ima određeno ponašanje samo u slučaju kada više parametara ima specifične vrednosti
    - npr. ako konačna ocena na ispitu zavisi od zbira bodova evidentiranih u različitim parametrima

Analiza graničnih slučajeva



Analiza graničnih slučajeva

- Dodatak metodi klasa ekvivalencije
- Bavi se vrednostima koje su na granicama klasa ekvivalencije
- Otkazi su vrlo često uzrokovani graničnim vrednostima
  - razlog je što često granice nisu jasno definisane ili su pogrešno interpretirane
- Tehnika se primenjuje kada su vrednosti u klasi ekvivalencije uređene i imaju jasno određene granice

Analiza graničnih slučajeva

- Za svaku klasu ekvivalencije testira se
  - vrednost koja je tačno na granici klase
  - najbliža vrednost graničnoj koja se nalazi izvan klase ekvivalencije
  - najbliža vrednost graničnoj koja se nalazi unutar klase ekvivalencije
- Susedna vrednost je vrednost sa minimalnim pomerajem u odnosu na graničnu vrednost
  - kod decimalnih brojeva, izabere se određena tolerancija
    - npr. sused vrednosti 1.00 je vrednost 1.01

Analiza graničnih slučajeva

- Ako se granična vrednost već nalazi u klasi, ne mora se testirati susedna vrednost koja je takođe unutar klase
  - u toj varijanti su dovoljne 2 vrednosti
    - granična vrednost koja je unutar klase i sused izvan klase
  - ovakav test je manje pouzdan jer pretpostavlja da se granična i susedna vrednost isto tretiraju

Primer analize graničnih slučajeva



- Primer
  - `students-testing/StudentServiceTest::testCalculateGrade()`

Dizajn test slučajeva

- Slično metodi ekvivalentnih klasa, identifikuju se
  - granične i susedne vrednosti za svaki parametar
  - sve kombinacije ovih vrednosti parametara
- Vrši se razuman izbor koji podskup mogućih test slučajeva treba kreirati
  - umesto svih mogućih kombinacija parametara, test slučajevi se kreiraju tako da se svaka izabrana vrednost koristi u bar jednom test slučaju
- Sused granične vrednosti jedne klase je često granična vrednost druge klase
  - testiranje te vrednosti je isti test slučaj
  - to smanjuje broj potrebnih test slučajeva

Kriterijum završetka

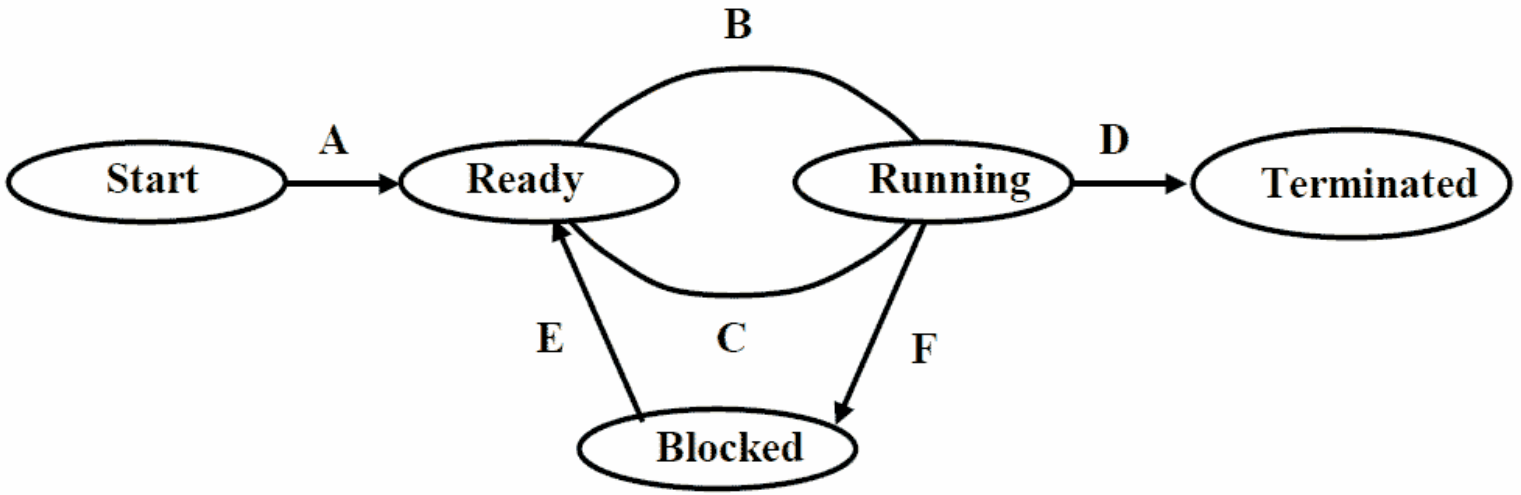
- Kriterijum završetka je postignuti nivo pokrivenosti svih graničnih vrednosti
- Računa se kao odnos testiranih i ukupnih graničnih vrednosti
- $coverage = \frac{testedBV}{totalBV} * 100\%$
- U ukupan broj graničnih vrednosti ulaze i najbliži susedi graničnih vrednosti
  - ako se ove vrednosti preklapaju za dve klase ekvivalencije, vrednost se računa samo jednom

Pregled korišćenja analize graničnih slučajeva

- Koristi se u kombinaciji sa metodom klasa ekvivalencije

- Poboljšava rezultate metode klasa ekvivalencije jer se greške češće nalaze na granicama nego unutar klase ekvivalencije

Testiranje prelaza stanja



Testiranje prelaza stanja

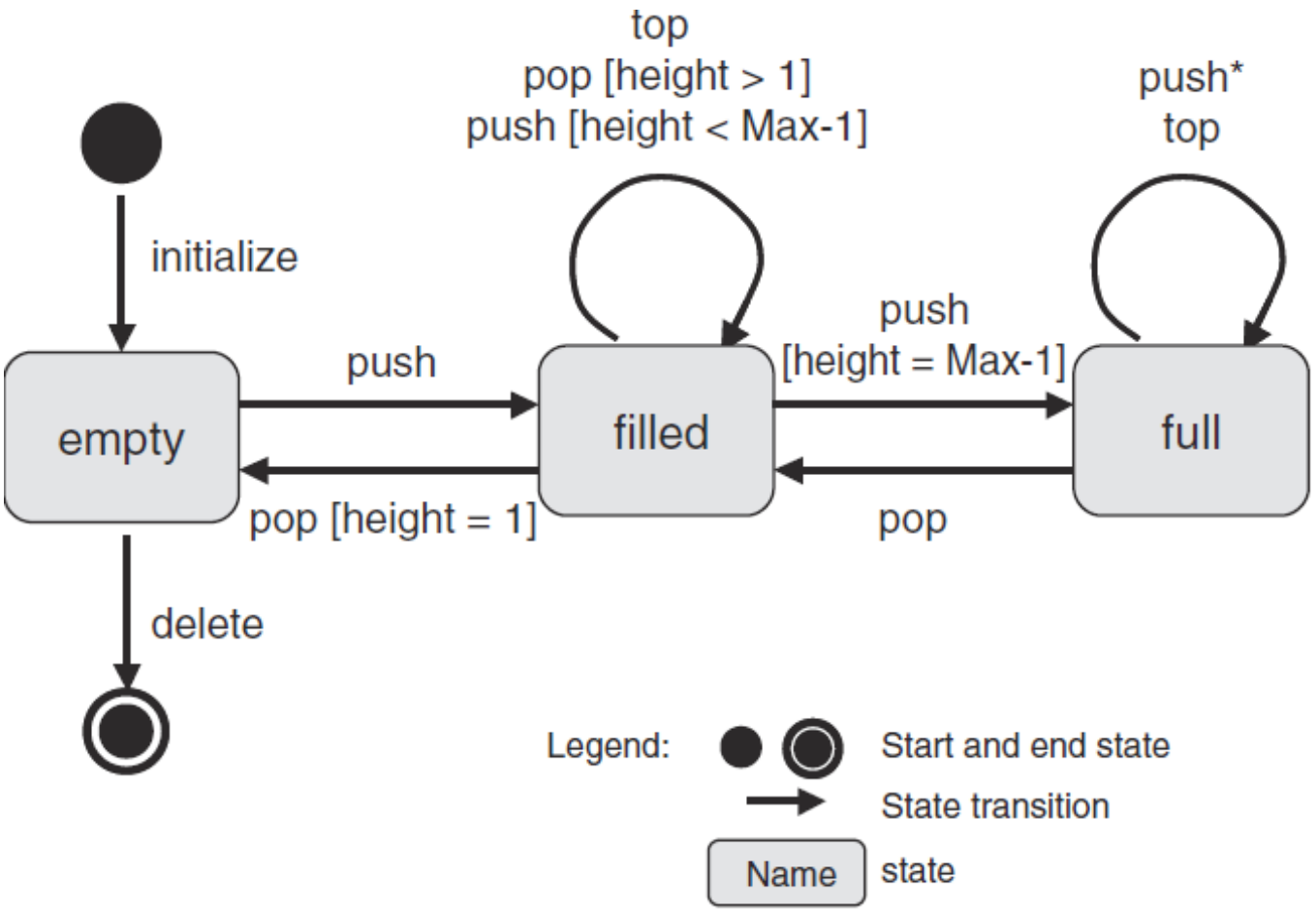
- Koristi se kada je važna istorija izršavanja test objekta
  - test objekat se verifikuje praćenjem stanja kroz koja prolazi u toku izvršavanja
  - sistem različito reaguje na akcije zavisno od trenutnog stanja
- Dijagramom prelaza stanja se ilustruju
  - stanja u kojima test objekat može da bude
  - akcije koje iniciraju prelaze stanja
- Izvršavanjem test objekta se verifikuje da li akcije prevode objekat u odgovarajuće stanje
  - da bi se od inicijalnog stanja došlo u neko stanje S potrebno je izvršiti niz prelaza stanja odgovarajućim akcijama

Testiranje prelaza stanja

- Posebno korisno kod *end-to-end* testova
  - Testira se interakcija sa korisnikom kroz grafički interfejs
  - Grafički interfejs se sastoji od niza prozora/stranica kroz koje korisnik prolazi
  - Određenim akcijama korisnik prelazi sa jedne stranice na drugu
  - Stranice su moguća stanja sistema, a korisnikove akcije su operacije koje sistem prevode iz jednog stanja u drugo

Primer dijagrama stanja

- Stek ograničenog kapaciteta



Primer dijagrama stanja

- Rezultat operacije zavisi od trenutnog stanja
  - npr. operacija *push* prevodi sistem u stanje *filled* ili *full*
  - određene operacija nisu dozvoljene u nekima od stanja
    - npr. *delete* ako stanje nije *empty*
  - dakle stanje test objekta je faktor koji utiče na to šta smatramo ispravnim ponašanjem
    - testiranje mora ovo da uzme u obzir

Primer test slučaja

- Obzirom da rezultat akcije zavisi od trenutnog stanja, veoma je važno definisati preduslov za test slučaj

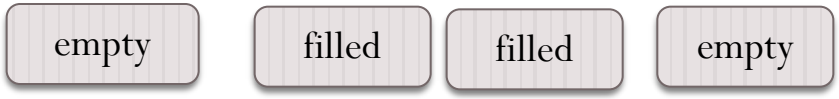
- Preduslov
  - Stek je kreiran, nalazi se u stanju *empty*
- Ulazna operacija
  - operacijom *push* šalje se na stek vrednost *Petar*
- Očekivani rezultat operacije
  - na stek je postavljena vrednost *Petar*
- Posledica
  - Stanje steka je *filled*

Intenzitet testiranja

- Minimalno
  - testiranjem dovesti sistem u sva moguća stanja
  - Npr. za stek sa mogućim stanjima *empty, filled, full* kapaciteta 4 elementa, izvršiti sledeće operacije
  - *initialize*, *push*, *push*, *push*, *push*



- U prethodnoj varijanti nisu testirane sve operacije
- Druga varijanta je da test izvrši sve operacije
  - initialize, push, top, pop, delete

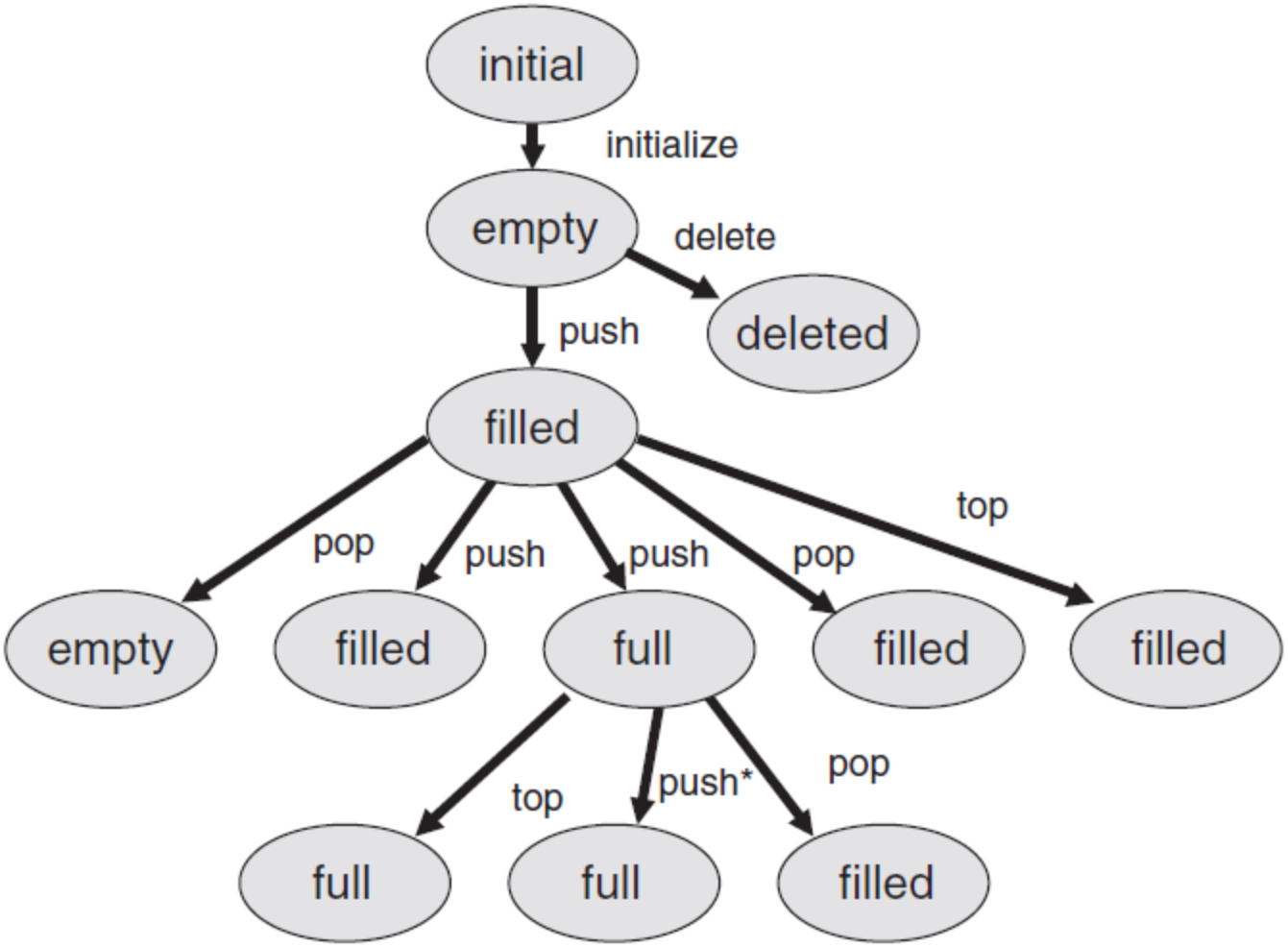


  - sada sistem nije doveden u sva moguća stanja

Intenzitet testiranja

- Test bi trebao da za svako stanje bar jednom izvrši svaku od funkcija mogućih u tom stanju
  - negativni testovi bi trebali da testiraju i izvršavanje funkcija koje nisu dozvoljene u tom stanju
- Za realizaciju ovakvog testa potrebno je transformisati dijagram prelaza stanja u stablo prelaza
- Stablo prelaza
  - sadrži konačnu necikličnu sekvencu prelaza stanja
  - sekvenca bar jednom sadrži sva moguća stanja i sve moguće prelaze stanja

Primer stabla prelaza

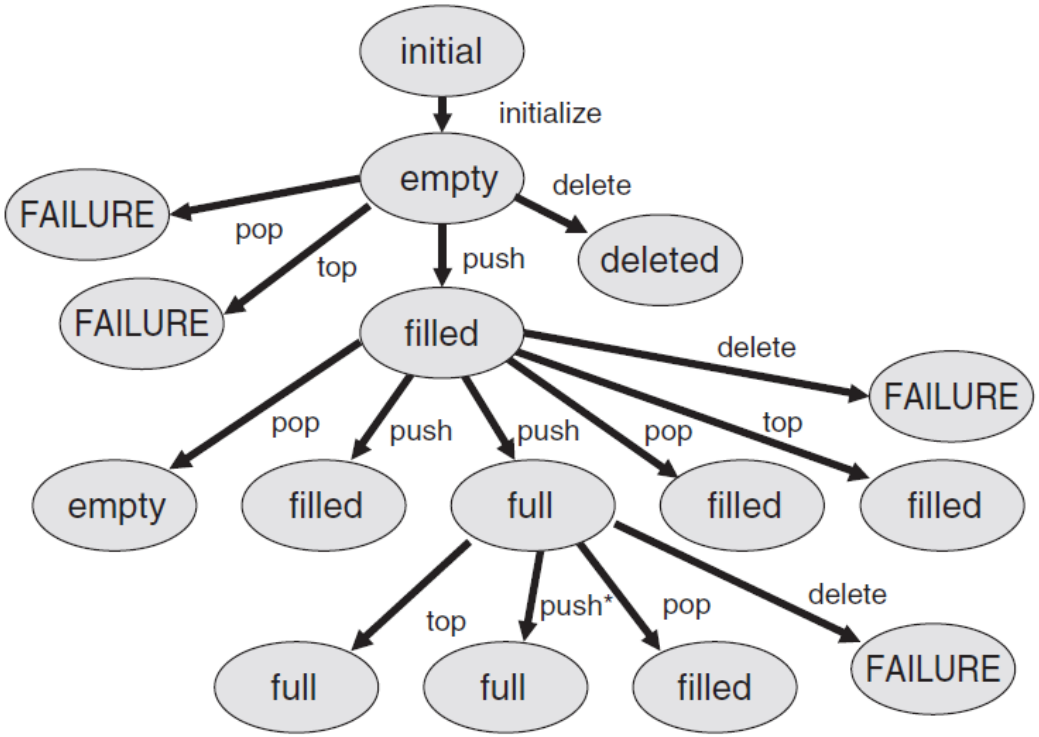


Kreiranje stabla prelaza

- Pravila za kreiranje stabla prelaza na osnovu dijagrama prelaza stanja
  1. Inicijalno stanje je koren stabla
  2. Za svaki mogući prelaz iz inicijalnog stanja u neko drugo stanje, na stablo se dodaje
     - čvor koji reprezentuje to stanje i
     - veza koja reprezentuje operaciju koja inicira prelaz u to stanje
  3. Korak 2 se ponavlja za nove čvorove
  4. Proces se završava kada
     - je triplet *{prethodno stanje, naredno stanje, operacija}* već kreiran u stablu
     - je stanje konačno stanje iz kojeg nije moguće više prelaziti u druga stanja

Kreiranje stabla prelaza

- Za negativne testove, u stablo treba uključiti i reakciju sistema na funkcije koje nisu dozvoljene u određenim stanjima



Kreiranje test slučajeva

- Test slučajevi se kreiraju na osnovu stabla prelaza
- Jedan test slučaj testira jednu putanju kroz stablo
- Za svaki prelaz stanja potrebno je
  - odrediti stanje pre prelaza
    - to je preduslov za test slučaj
    - osim za inicijalno stanje
  - događaj/operacija koja inicira prelaz stanja
  - očekivanu reakciju sistema na operaciju
  - naredno očekivano stanje
    - to je posledica operacije

Kriterijum završetka

- Manje intenzivna varijanta
  - Svako stanje je bar jednom dostignuto
  - Svaka operacija je bar jednom izvršena
  - Svaka nedozvoljena operacija je bar jednom izvršena
- Intenzivna varijanta za visokokritične aplikacije
  - sve kombinacije prelaza stanja su izvršene
  - svi prelazi stanja u svim mogućim redosledima uključujući uzastopna izvršavanja istih prelaza

Logički zasnovane tehnike



Logički zasnovane tehnike

- Posmatraju i međusobne zavisnosti između ulaznih parametara
  - za razliku od drugih tehnika koje pri kreiranju test slučajeva svaki parametar nezavisno posmatraju

- Različite logički zasnovane tehnike
  - tehnika uzročno-posledičnih grafova
  - tehnika parova podataka

Uzročno-posledični graf

- Graf koji prikazuje vezu između uzroka i posledice u sistemu
- Prikazuju se uslovi koji trebaju da budu ispunjeni da bi sistem dao određeni rezultat
- Između uslova se prave logičke veze
  - „i“, „ili“, „ne“

Primer uzročno-posledičnog grafa

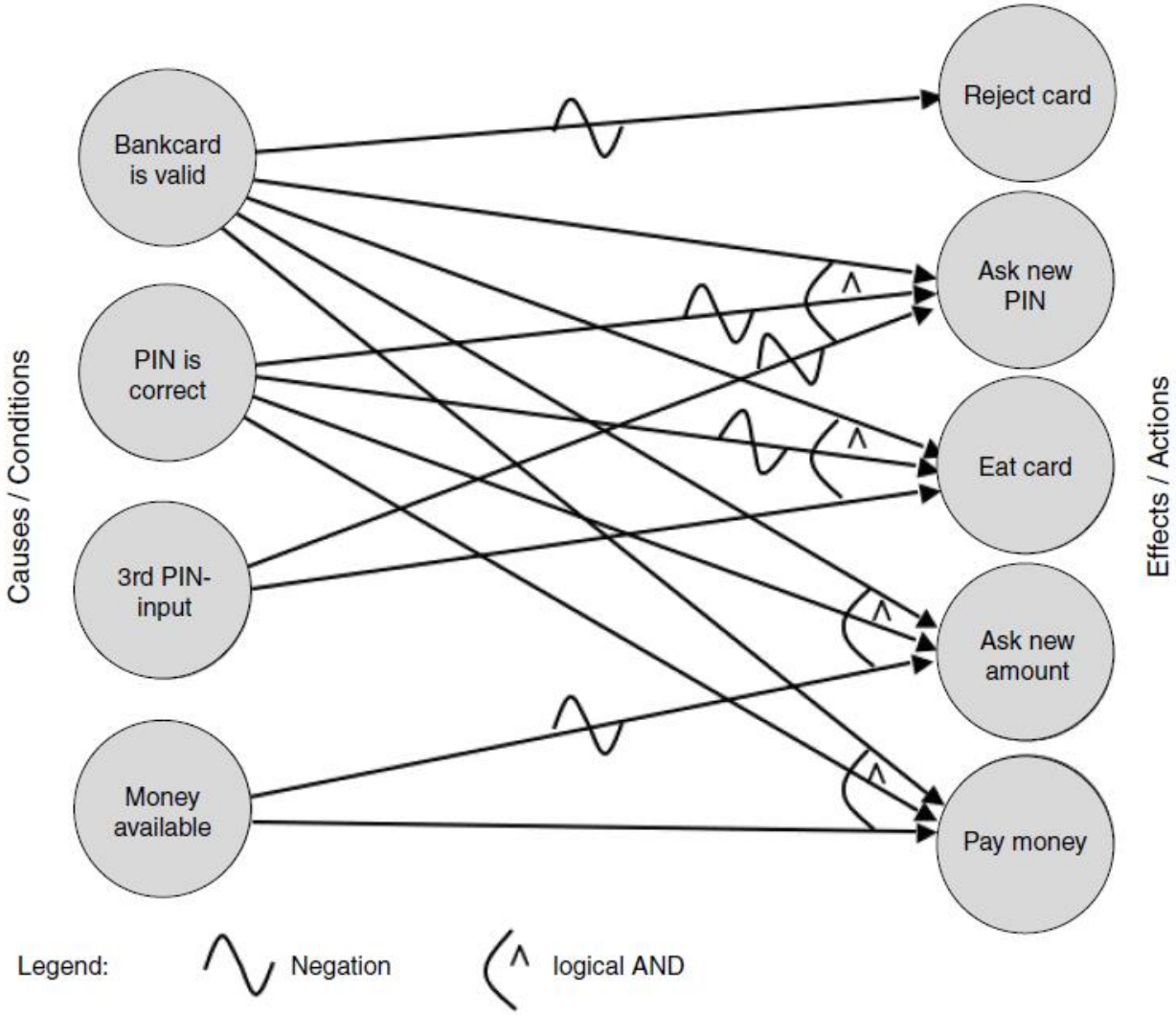


Tabela odlučivanja

- Za kreiranje test slučajeva, potrebno je transformisati uzročno-posledični graf u tabelu odlučivanja
- Tabela odlučivanja
  - sadrži informacije o pravilima po kojima određena kombinacija ulaznih parametara daje očekivane izlaze

Primer tabele odlučivanja

Decision table		TC1	TC2	TC3	TC4	TC5
Conditions	Bank card valid?	N	Y	Y	Y	Y
	PIN correct?	-	N	N	Y	Y
	Third PIN attempt?	-	N	Y	-	-
	Money available?	-	-	-	N	Y
Actions	Reject card	Y	N	N	N	N
	Ask for new PIN	N	Y	N	N	N
	"Eat" card	N	N	Y	N	N
	Ask for new amount	N	N	N	Y	N
	Pay cash	N	N	N	N	Y

- Gornji redovi su uzroci, a donji su posledice
- Svaki test slučaj podrazumeva određenu kombinaciju uzroka koji daju jednu posledicu
- Neke kombinacije nemaju smisla pa se ne analiziraju
  - npr. vrednosti drugih parametara ako je kartica neispravna

Kreiranje test slučajeva

- Test slučajevi se kreiraju na osnovu tabele odlučivanja
  - pri kreiranju tabele odlučivanja, treba krenuti od izlaza da bi se identifikovalo koji očekivani izlaz test slučaj verifikuje
  - svaki uzrok i posledica trebaju u tabeli bar jednom da imaju vrednost „da“ i „ne“
- Svaka kolona u tabeli je jedan test slučaj
  - prateći vrednosti u koloni vide se zahtevane vrednosti ulaznih parametara kao i očekivani izlaz
  - konkretne reprezentativne vrednosti nisu navedene u tabeli odlučivanja
    - određuju se pri dizajnu test slučaja

Pregled tehnike uzročno-posledičnih grafova

- Sistematično evidentiranje veza između uzroka i posledica u sistemu može da otkrije kombinacije koje nisu bile analizirane drugim testovima
- Formiranje ovakve evidencije je dosta zahtevno za komplikovanije sisteme
  - neke kombinacije se ne unose u tabelu odlučivanja što može greškom dovesti do izbacivanja i kombinacija koje treba verifikovati
  - broj test slučajeva može da poraste toliko da ih nije moguće implementirati

Tehnika parova podataka

- Koristi se kada međusobna zavisnost pojedinih parametara nije poznata
  - za razliku od tehnike uzročno-posledičnih grafova koja analizira poznate zavisnosti
- Cilj je pronaći da li između parametara postoje zavisnosti koje ugrožavaju rad sistema
  - koristi se za verifikaciju pretpostavke da su određeni parametri nezavisni i da njihova interakcija ne daje štetne efekte

Tabela kombinacija

- Tehnika parova podataka polazi od klasa ekvivalencije
- Za svaki parametar se iz svake klase ekvivalencije bira reprezentativna vrednost
- Pravi se tabela kombinacija reprezentativnih parametara
  - ali samo kombinacije po parovima
  - tabela uključuje sve moguće kombinacije između svaka dva parametra

Primer tabele kombinacija

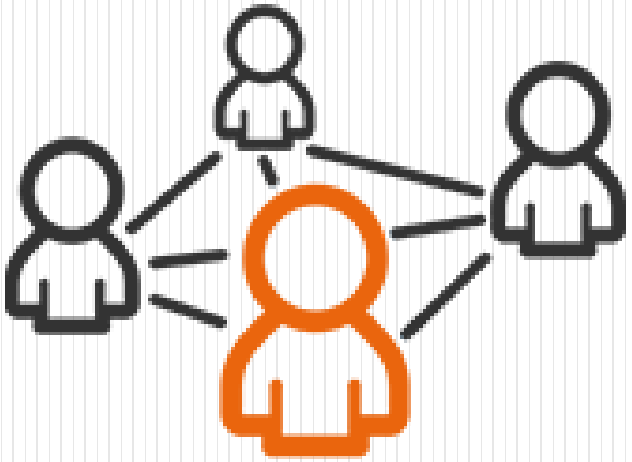
Test case #	OS	Language	Screen
1	Mac	German	small
2	Linux	German	large
3	Windows	German	large
4	Mac	Norwegian	large
5	Linux	Norwegian	small
6	Windows	Norwegian	small
7	Mac	English	large
8	Linux	English	small
9	Windows	English	Choose freely

- Ako odvojeno posmatramo bilo koji par parametara, sve kombinacije su uključene
- Nije svaki mogući triplet uključen
  - npr. ne postoji kombinacija *{Mac, German, large}*

Kreiranje test slučajeva

- Svaki red tabele kombinacija je jedan test slučaj
  - uključuje verifikaciju jedne kombinacije vrednosti parova
- Na ovaj način pronaći ćemo svaki nedostatak uzrokovan štetnom interakcijom između neka dva parametra
  - preciznije između svake od klasa ekvivalencija ovih parametara
- Tehnika nije namenjena otkrivanju nedostataka koje uzrokuje interakcija više od dva parametra

Testiranje zasnovano na slučajevima korišćenja



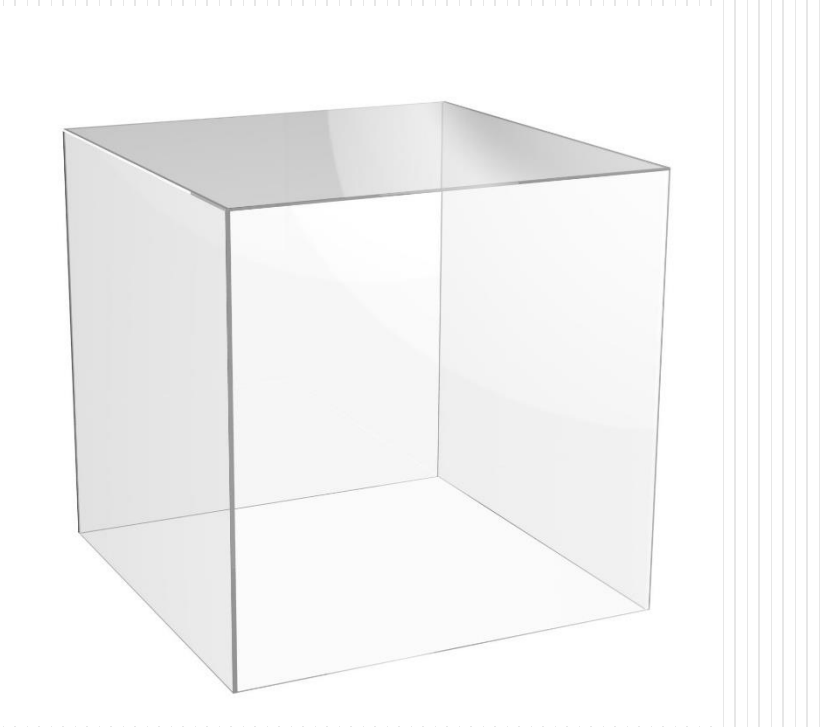
Testiranje zasnovano na slučajevima korišćenja

- Zahtevi se predstavljaju i dijagramima slučajeva korišćenja
- Testovi koji verifikuju funkcionalnosti se mogu izvesti iz ovih dijagrama
- Obzirom da dijagrami modeluju pogled na sistem iz ugla krajnjeg korisnika, ova tehnika je korisna za *end-to-end* i *acceptance* testove

Kreiranje test slučajeva

- Izvode se iz slučajeva korišćenja
- Slučaj korišćenja ima
  - cilj i treba da obezbedi određene rezultate
  - preduslove za izvršavanje
  - postuslove
- Konkretne ulazne i očekivane rezultate određuje test slučaj
  - nisu izraženi slučajem korišćenja
- Treba verifikovati i sve alternativne putanje (moguća proširenja) slučaja korišćenja
  - izražene *extend* vezom u UML dijagramu

White box testiranje



White box testiranje

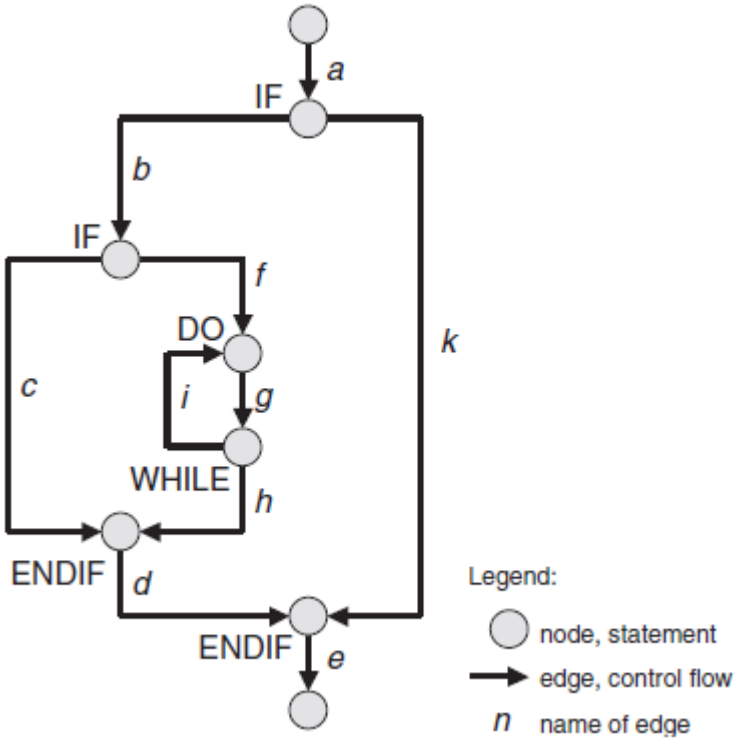
- Osnova za testiranje je izvorni kod test objekta
  - zove se i testiranje zasnovano na strukturi ili kodu
- Uzima u obzir koji delovi izvornog koda test objekta se izvršavaju pri testiranju
  - dobar test bi trebao da izvrši sve delove izvornog koda bar jednom
- Pri dizajnu test slučajeva uzima se u obzir koji delovi koda će se izvršiti pri pokretanju test slučajeva
- Očekivani rezultat se određuje na osnovu specifikacije zahteva, a ne analizom koda
  - jer kod može da ima grešku, a upravo je svrha testiranja da se te greške pronađu

Tehnike *white-box* testiranja

- Testiranje naredbi
- Testiranje grana
- Testiranje uslova
  - jednostavno testiranje uslova
  - višestruko testiranje uslova
  - minimalno višestruko testiranje uslova
- Testiranje putanja

Analiza izvornog koda

- Za lakšu analizu izvornog koda, kod se transformiše u graf toka programa
- Naredbe programa su čvorovi grafa
- Tok programa je predstavljen granama grafa
- Sekvenca naredbi koje bezuslovno slede jedna drugu se predstavlja kao jedan čvor



Testiranje naredbi



Testiranje naredbi

- Analizira se svaka naredba u izvornom kodu test objekta
  - test slučajevi treba da izvrše određen procenat svih naredbi (ili sve naredbe)
  - pokrivenost testa (coverage) utvrđuje koliko je naredbi izvršeno testovima
  - $(broj\ izvršenih\ naredbi\ /\ ukupan\ broj\ naredbi)\ *100\%$
  - naziva se i C0 (C-zero) pokrivenost
- Za graf sa slike
  - dovoljan je jedan test slučaj da se izvrše sve naredbe
  - to je test slučaj sa tokom **a, b, f, g, h, d, e**

Testiranje naredbi

- Fokusiranje samo na pokrivenost svih naredbi ne pokriva dovoljno moguće tokove programa
  - ne pokriva izvršavanje grana koje nemaju naredbe
- Ipak, čak i za ovakve testove je teško obezbediti 100% pokrivenost
  - Neke izuzetke u kodu je teško izazvati

Testiranje grana



Testiranje grana

- Naprednija tehnika od testiranja naredbi
- Analiziraju se grane u grafu toka programa
- Zavisno od uslovnih izraza, prati se koje grane toka programa se izvršavaju
- Pokrivenost se određuje u odnosu na procenat grana izvršenih u okviru testa
  - $(broj\ izvršenih\ grana\ /\ ukupan\ broj\ grana) * 100\%$
  - naziva se i C1 pokrivenost
  - za razliku od testiranja naredbi, potrebno je pokriti i grane bez naredbi
    - npr. `if` naredbu bez `else` dela

Testiranje grana

- Pri dizajnu test slučajeva vodi se računa o svim granama toka programa
- Za graf sa slike neophodna su tri test slučaja za pokrivenost 100%
  - **a, b, c, d, e**
  - **a, b, f, g, i, g, h, d, e**
  - **a, k, e**
- Neke grane se izvršavaju više puta
  - jeste redundantno, ali je neophodno jer različiti tokovi imaju neke zajedničke delove

Testiranje grana

- Predstavlja strožiji kriterijum testiranja u odnosu na testiranje naredbi
  - obično zahteva kreiranje više test slučajeva
  - može da utvrdi nedostajuće naredbe u praznim granama
  - opštija tehnika od testiranja naredbi
    - potpuna pokrivenost grana garantuje i potpunu pokrivenost naredbi, ali ne važi obrnuto
- Testiranje grana ne analizira koliko puta se grana izvršava
  - pokrivenost se meri samo na osnovu toga da li je grana uopšte izvršena

Testiranje uslova



Testiranje uslova

- Analiziraju se konkretni uslovni izrazi koji uzrokuju prelazak toka programa na određenu granu
- Za razliku od testiranja grana koje se fokusira na proveru grana, testiranje uslova se bavi time šta dovodi do toga da program pređe u određenu granu

- Uslov može biti složen
  - više prostih uslova povezanih logičkim operatorima
  - prost uslov je onaj koji
    - daje logičku vrednost kao rezultat i
    - ne sadrži logičke operatore
    - sadrži relacione operatore koji daju logički rezultat

Jednostavno testiranje uslova

- Kreiraju se testovi tako da se za svaki prost uslov proveri ponašanje programa za oba moguća rezultata (i *true* i *false)*
- Primer uslova

  (x > 3 || y < 5)
- Test slučaj 1
  - x je 6, y je 8
  - Prvi prost uslov je *true*, drugi je *false*
- Test slučaj 2
  - x je 2, y je 3
  - Prvi prost uslov je *false*, drugi je *true*

Jednostavno testiranje uslova

- Sa ova dva test slučaja ispunjen je zahtev jednostavnog testiranja uslova
  - oba prosta uslova su pri testiranju imala rezultate i *true* i *false*
- Rezultat kompletnog složenog uslova je u oba test slučaja *true*
- Ovakvo testiranje će proveriti samo jednu od dve moguće grane toka programa
  - ovakav kriterijum testiranja je blaži u odnosu na testiranje grana, jer ne mora proveriti sve grane
  - ne gleda se rezultat kompletnog uslova nego samo rezultati prostih uslova

Višestruko testiranje uslova

- Zahteva da testovi provere sve moguće kombinacije rezultata prostih uslova
  - za razliku od jednostavnog testiranja uslova koje ne gleda kombinacije nego samo rezultat pojedinačnog prostog uslova
- Za uslov (x > 3 || y < 5) trebaju 4 test slučaja
  - x je 6, y je 3, *true* || *true*, rezultat je *true*
  - x je 6, y je 8, *true* || *false*, rezultat je *true*
  - x je 2, y je 3, *false* || *true*, rezultat je *true*
  - x je 2, y je 8, *false* || *false*, rezultat je *false*

Višestruko testiranje uslova

- Ovaj kriterijum testiranja sigurno pokriva sve moguće grane
  - ako proverimo sve moguće kombinacije uslova, svaka grana će biti pokrivena nekom od kombinacija
- Ovo je najstrožiji kriterijum, ali težak za implementaciju
  - za komplikovanije uslove postoji prevelik broj mogućih kombinacija
    - 2^n ako je n broj prostih uslova

Višestruko testiranje uslova

- Nekada ne postoje ulazni podaci za određene kombinacije uslova
- Primer za uslov (x ≥ 3 && x < 5)
  - ne postoji vrednost x koja za oba prosta uslova daje *false*
    - ne postoji vrednost koja je istovremeno manja od 3 i veća ili jednaka 5

Minimalno višestruko testiranje uslova

- Optimizacija višestrukog testiranja uslova
- Ne proveravaju se sve moguće kombinacije prostih uslova
- Testovi uključuju samo one moguće kombinacije u kojima promena rezultata prostog uslova menja rezultat složenog uslova
  - time se verifikuje da li je prost uslov dobro implementiran
  - ako nije dobro implementiran, složeni uslov će dati drugačiji rezultat

Minimalno višestruko testiranje uslova

- Potrebno je manje test slučajeva nego kod klasičnog višestrukog testiranja uslova
- Za uslov (x > 3 || y < 5) trebaju 3 test slučaja
  - x je 6, y je 8, *true* || *false*, rezultat je *true*
  - x je 2, y je 3, *false* || *true*, rezultat je *true*
  - x je 2, y je 8, *false* || *false*, rezultat je *false*
- Nema potrebe za test slučajem
  - x je 6, y je 3, *true* || *true*, rezultat je *true*
  - Ako je jedan od dva prosta uslova netačno implementiran i da rezultat *false*, i dalje će vrednost kompletnog uslova biti *true*

Pregled testiranja uslova

- Pri dizajnu test slučajeva analizira se koji ulazni podaci daju koje rezultate i kombinacije prostih uslova
- Pokrivenost se računa kao procenat izvršenih u odnosu na sve moguće logičke vrednosti uslova
- Preporučeni način testiranja uslova je kroz minimalno višestruko testiranje uslova
  - uzima u obzir kompleksnost uslova
  - razuman broj test slučajeva
  - ne daje manju pokrivenost od testiranja izraza i grana

Pregled testiranja uslova

- Voditi računa da uslov može da sadrži logičke promenljive koje su ranije u kodu dobile rezultat nekog složenog uslovnog izraza
  - test ne bi trebao da gleda samo uslov u kojem se rezultat promenljive koristi, nego i ranije dodele vrednosti logičkim promenljivim
- Voditi računa o načinu evaluacije uslova pri izvršavanju programa
  - Ne proverava se prost uslov čiji rezultat ne utiče na konačan rezultat uslova
    - npr. drugi operand u konjukciji, ako je prvi *false*

Testiranje putanja



Testiranje putanja

- Uzima u obzir sve različite putanje u toku programa, uključujući višestruko izvršavanje istih grana
  - analizira i ponovno izvršavanje grana u petlji
- Npr. za graf toka programa iz primera tok bi mogao da bude
  - **a, b, f, g, i, g, i, g, i, g, i, g, h, d, e**
- Zbog velikog broja mogućih putanja (iteracija u petlji), ne treba zahtevati 100% pokrivenost kod ovog tipa testiranja
  - ali u svakom slučaju testiranje mora uzeti u obzir da li ponovna izvršavanja grana utiču na ispravnost programa

Testiranje zasnovano na iskustvu i intuiciji



Testiranje zasnovano na iskustvu i intuiciji

- Umesto sistematskog pristupa, test slučajevi se intuitivno dizajniraju
  - na osnovu iskustva i intuicije testera
- Koristi se kao podrška sistematskim tehnikama
  - sistematski pristup sledi definisana pravila i može da previdi neke nedostatke
  - iskustvo pomaže u sagledavanju šire slike
- Testovi pokrivaju delove aplikacije za koje tester proceni da mogu da kriju nedostatke
  - na osnovu grešaka iz prethodnih projekata, problema u tehnologiji koja se koristi, ...
- Nije moguće meriti intenzitet i pokrivenost testiranja
  - nema formalnog kritetijuma završetka

Istraživačko testiranje

- eng. *exploratory testing*
- Jedna od tehnika testiranja zasnovana na iskustvu i intuiciji
- Koristi se kada je specifikacija nepotpuna ili ne postoji
  - nije moguće sistematskim tehnikama testirati usaglašenost sa specifikacijom
- Ne koristi se klasično planiranje procesa testiranja
  - minimalno trošenje resursa na planiranje i pripremu
- Elementi test objekta se istražuju
  - nakon toga se donosi odluka koje elemente treba testirati

Istraživačko testiranje

- Izvršavanjem test slučajeva se istražuje kako se test objekat ponaša
  - ovim se otkrivaju „interesantni“ delovi za testiranje kroz nove test slučajeve
- Istraživačko testiranje objedinjuje
  - učenje o test objektu
  - dizajniranje test slučajeva
  - izvršavanje test slučajeva
- Ovaj pristup daje testeru najviše slobode
  - potrebno je iskustvo da bi pristup dao rezultate
  - tester formira mentalni model aplikacije
    - šta aplikacija radi i šta bi trebala da radi

Istraživačko testiranje

- Najčešće se fokusira na određeni deo programa
  - deo se naziva *test charter*
  - najviše jedan ili dva sata testiranja za jedan *charter*
- Planiranje je minimalno
- Formira se lista za taj *test charter*
  - Cilj testiranja *chartera*
  - Šta treba da bude testirano
  - Na koji način će testiranje biti izvršeno
  - Kakvi se problemi očekuju

Istraživačko testiranje

- Sve stavke su okvirno opisane, testeru su prepušteni detalji
- Primer stavke
  - „*Istražiti neuspešno logovanje korisnika. Proveriti reakciju sistema na zaboravljenu lozinku i resetovanje lozinke.*“
  - Tester će istražiti da li sistem zabranjuje logovanje sa neodgovarajućim kredencijalima i kako registrovani korisnik može da dobije ili resetuje lozinku

Merenje pokrivenosti koda u Java aplikaciji



JaCoCo



- JaCoCo je biblioteka za merenje pokrivenosti Java koda testovima
- http://www.eclemma.org/jacoco/
- Meri pokrivenost
  - naredbi (C0) i grana (C1)
  - metoda
  - klasa

Korišćenje JaCoCo biblioteke u Spring aplikaciji

- Potrebno je dodati JaCoCo u Maven zavisnosti

```
<dependency>
   <groupId>org.jacoco</groupId>
   <artifactId>jacoco-maven-plugin</artifactId>
   <version>0.8.10</version>
</dependency>
```

Korišćenje JaCoCo biblioteke u Spring aplikaciji

- Potrebno je konfigurisati da se pri izgradnji projekta aktivira JaCoCo plugin

```
<plugin>
   <groupId>org.jacoco</groupId>
   <artifactId>jacoco-maven-plugin</artifactId>
   <version>0.8.10</version>
   <executions>
   ...
```

Korišćenje JaCoCo biblioteke u Spring aplikaciji

- Pokretanjem Maven izgradnje pokreće se i JaCoCo plugin koji generiše izlazni JaCoCo fajl sa podacima o pokrivenosti koda
  - `mvn clean test`
  - dobija se `target/jacoco.exec`

- Ovo je binarni fajl koji je potrebno procesirati nekim alatom da bismo dobili podatke čitljive za čoveka

Korišćenje JaCoCo biblioteke u Spring aplikaciji

- Sonar može da procesira JaCoCo izlazne fajlove
- Potrebno je pokrenuti sonar cilj korišćenjem maven alata
  - `mvn sonar:sonar -Dsonar.token=abcdefghijkl`
  - pokreće se iz korenskog foldera projekta
- Kroz Sonar web GUI se mogu videti podaci o pokrivenosti koda
  - na http://localhost:9000 za lokalno instalirani Sonar